Implementation Reference

Context-aware visual anomaly detection for drone and CCTV footage. This document describes what the system does, how each stage is built, why it is built that way, and how to run it end to end.

Companion documents: `CLAUDE.md` (hardware and path constraints), `HANDOVER.md` (live competition state), `PROGRESS.md` (dated build log).

---

1. The problem, stated precisely

An anomaly here is not a visual category. It is a **context violation**.

A parked car and a car stopped dead in a live highway lane are, in a single frame, the same arrangement of pixels. What separates them is *where* the vehicle is and *how long* it has been there. This has one hard consequence that shapes the entire architecture:

> **A still frame does not contain motion.** Any component that sees only one image is structurally incapable of distinguishing "stopped" from "moving", no matter how large or well-trained it is.

Everything below follows from taking that seriously. The system is required to classify twelve anomaly types, localise them in time, run in real time, and do so on a 4GB consumer GPU.

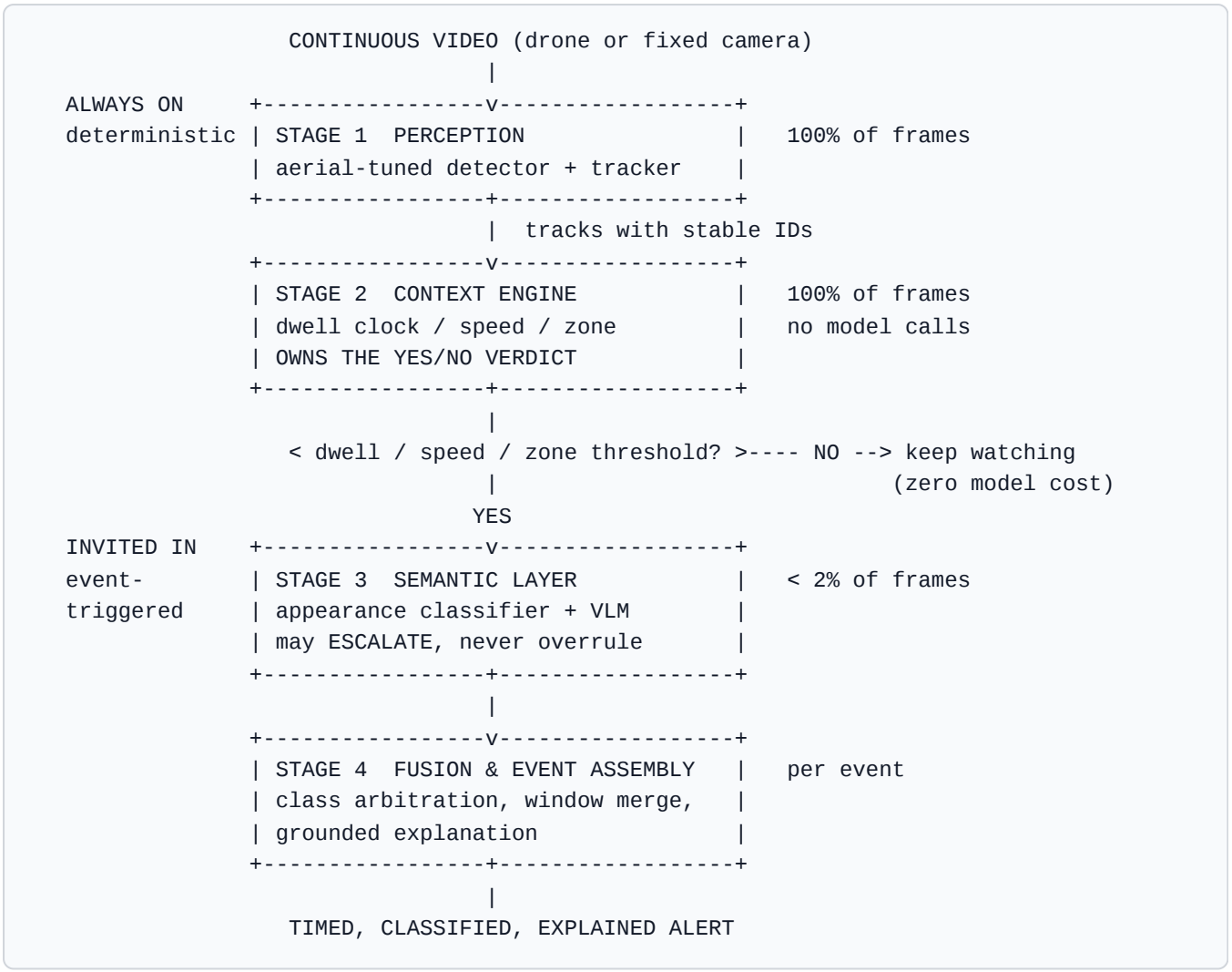
Anomaly vocabulary

```
normal                          traffic_accident
traffic_congestion              stalled_or_broken_down_vehicle
vehicle_blocking_traffic        wrong_way_driving
road_spill_or_debris            waterlogging_or_flood
fire                            smoke
fighting_or_violence            loitering_or_suspicious_presence
```

---

2. Architecture

Four stages and one gate. The gate is the load-bearing idea: it separates work that must happen constantly from work that is too expensive to happen constantly.



Why this split

The division of labour was decided by measurement, not preference. Asked directly whether a scene was anomalous, a 3B vision-language model scored at chance (3/6) across four prompt revisions on a clip with a known stopped truck — while its *prose* described the situation correctly every time. It tracked whichever rule the prompt emphasised rather than the image.

The tracker, meanwhile, knows the dwell time exactly. So:

- **Stage 2 rules own the boolean.** Dwell, speed and zone are arithmetic.
- **Stage 3 only observes.** Descriptive tasks are what these models do well.
- **Stage 3 may escalate, never clear.** A visible hazard promotes an event's severity. A model saying "nothing visible" cannot cancel a stop the tracker measured. Evidence accumulates in one direction only, so confirmed catches are structurally safe.

---

3. Stage 1 — Perception

Files: `src/detector.py`, `src/tracker.py`, `src/run_ahc_dataset.py`

- Detector: YOLO (Ultralytics), retrained on aerial/overhead footage. Off-the-shelf weights are trained on eye-level photography and miss most objects viewed from altitude; retraining roughly tripled recall on this viewpoint.
- Tracker: ByteTrack via `supervision`, giving each object an identity that persists across frames. Stage 2 is meaningless without stable IDs — a dwell clock needs to know it is timing the same vehicle.
- **Threaded decode.** 4K decode (23.5 ms/frame) and detection (29.3 ms/frame) are serial-additive if run naively: 20.0 fps. Overlapping them via `iter_frames_threaded` hides the decode entirely: **26.6 fps, 1.06x real-time on 4K**. Do not revert this.

Ego-motion compensation

A drone drifts, so a stationary car translates across the image. Frame-to- frame scene motion is estimated by optical flow and cancelled before any measurement is taken, which converts every downstream number from a statement about the camera into a statement about the world.

---

4. Stage 2 — Context engine

Files: `src/rules.py`, `src/calibrate_zones.py`

No model calls. Per tracked identity it maintains:

signal	meaning
dwell / `age_s`	seconds this identity has stayed effectively stationary
normalised speed	displacement per frame in body-lengths, ego-motion corrected
zone	`driving_lane`, `shoulder`, `parking`, … from one-time calibration
neighbour state	whether surrounding traffic is still flowing

Zones come from a **one-time per-video calibration** (`calibrate_zones.py --auto`), not per-frame inference, writing `<video>_zones.json` beside the clip and reusing it on later runs.

A stop only counts once an object has been *observed moving first* (`collision_min_moving_s`). Without that guard, vehicles that were parked before the clip began fired as collisions at `age_s 0.0`.

A measured negative result

Speed-based congestion detection does not work on this footage and was removed. On a 256x192 clip, box jitter on a few-pixel vehicle swamps real speed: the *normal* clip T003 reads as more congested than both genuine congestion clips at every threshold from 0.05 to 0.50. No cutoff separates them. Congestion is handled by appearance instead. (`src/diag_speeds.py`)

---

5. Stage 3 — Semantic layer

5a. Appearance classifier

File: `src/appearance_classifier.py` · **weights:** `appearance11.pt`

An 11-class image classifier over sampled frames, covering the categories a single photo genuinely can show. It deliberately **excludes** `stalled_or_broken_down_vehicle` (too few training videos to learn), which the rules cover instead.

Its second job is temporal: `windows_for_label(video, cls)` samples a uniform time grid, scores each frame, and groups positive hits into intervals. This is what produces start/end times for long videos.

Decision rule (threshold 0.15, top-k 3, margin 0.10, normal-scale 0.5) was chosen by optimising the real scorer over dumped probabilities (`src/tune_appearance.py`), not guessed.

> **Per-class thresholds were rejected.** They reach in-sample macro-F1 0.289, but leave-one-video-out gives 0.230 against the global rule's 0.256 — with 11 thresholds and 1–4 ground-truth videos per class they memorise the visible set. Building the cross-validation check is what caught this.

5b. Fine-tuned vision-language model

Base: Qwen2.5-VL-3B-Instruct, 4-bit · **Method:** LoRA via Unsloth on Kaggle T4 · **Local runtime:** Ollama

Emits a strict JSON schema:

```
{"is_anomaly": true, "class_name": "traffic_accident",
 "description_summary": "one short sentence"}
```

Training data is built by `src/build_rich_vlm_dataset.py` from ground-truth intervals: 4670 train / 580 val samples over 6185 JPEGs, class-balanced by trimming the majority and oversampling starved classes, with a per-class frame budget that gives rare classes more frames.

Local VLM inference cannot run concurrently with the detector — one 4GB card cannot hold both, and warm latency is 27–45s per call. For a strict real-time demo use `--decision rules` ; `hybrid` is for enrichment or a second GPU. Bulk evaluation runs on Kaggle instead (§8).

---

6. Stage 4 — Fusion and event assembly

Files: `src/attach_l23_times.py` , `src/export_arena.py` , `src/explain_events.py`

This stage turns per-frame evidence into the events a scorer or operator sees. Three mechanisms, each added to fix a measured structural failure.

6a. Window geometry

Three bugs, all fixed, none fitted to any single clip:

1. **Windows were the span of sample times, not of the event.** A positive hit at time *t* only establishes that the event covers *t*, so windows are now padded by ±(sample interval)/2. This alone converted four near-misses (IoU 0.23–0.38) into four matches on one clip.
2. **The merge gap was a fixed 8–20 s,** which merged distinct ground-truth events sitting 5–20 s apart — and a merged window then fails the IoU≥0.5 gate against *both*. It is now a multiple of the actual sample interval, the only defensible scale.
3. **Minimum/maximum span grew or truncated from the group start,** anchoring windows to their first hit. Both now resize about the window **centre**.

Sampling density matters as much as geometry: a flat 64 frames over a 629 s clip is ~10 s resolution, which cannot localise a 2.6 s event at IoU 0.5 regardless of classifier quality. Default is now `--sample-dt 3.0 --max-frames 160`.

6b. Fragment merge (second pass)

The merge inside `windows_for_label` groups on *raw hit* spacing and then pads each group. Two groups that were far apart as raw hits can end up adjacent once padded, and nothing re-checked the now-smaller gap. A second, deliberately more aggressive pass runs over the padded windows. Measured: one clip fell from 8 windows for 2 real events to 3, cutting that level's false alarms from 13 to 8.

6c. Class arbitration

Prior, measured on ground truth: of the 8 anomalous long videos, **7 contain exactly one distinct class**. Emitting four classes for one clip therefore guarantees at least three are false alarms.

`--class-rel` drops any class whose best detector confidence falls below a fraction of the strongest class for that video; `--max-classes` caps the count outright. On eval clip E021 this removed `road_spill_or_debris` (0.49) while keeping `wrong_way_driving` (0.87), taking that video from 22 events to 11.

> This ranks *between* classes, where separation is wide. An earlier gate that ranked *within* a class by confidence was measured harmful — it deleted correct windows and cost 6.5 marks — and is disabled by default (`--rel-conf 0`). Confidence discriminates classes; it does not rank windows.

6d. Grounded explanations

`explain_events.py` composes each explanation from facts the pipeline actually measured — dwell time, zone, neighbouring traffic, window position — rather than from model prose. Guards, each added after reading bad output:

- `CLASS_RULES` gates tracker evidence on class consistency, so a wrong-way event never cites collision logic.
- `MACHINE_MARKERS` strips tool-generated phrases ("confidence 0.42").
- `NORMALITY_MARKERS` / `CLASS_KEYWORDS` drop appended captions that contradict the claimed anomaly.
- Dwell is only cited when ≥5 s, so it represents real persistence.

Training captions cannot be used directly: 4670 rows contain only **333 distinct** captions, the most common repeated 400 times. Templated output is a property of the data, not a prompting bug — no decoding change fixes it.

---

7. Running the pipeline

Interpreter is always `C:\dvad\.venv\Scripts\python.exe`. Every script in `src/` runs standalone and takes `--data_dir`; a new dataset is a one-flag swap.

```
# Stage 1-2 (+ appearance), per level
python src\run_ahc_dataset.py --data_dir C:\dvad\data\<set>\L1 --split test --level 1 `
    --label-source hybrid --appearance-weights C:\dvad\models\appearance11.pt `
    --out C:\dvad\outputs\pred_L1.csv

# Stage 4: timing for the long levels
python src\attach_l23_times.py --pred C:\dvad\outputs\pred_all.csv `
    --videos <video_dir> --manifest <manifest.json> --class-rel 0.75 `
    --out C:\dvad\outputs\pred_timed.csv

# Export to submission JSON (validates schema)
python src\export_arena.py --manifest <manifest.json> --pred
C:\dvad\outputs\pred_timed.csv `
    --summaries C:\dvad\outputs\ahc_events --videos <video_dir> `
    --out C:\dvad\outputs\submission.json

# Add fact-grounded explanations
python src\explain_events.py --sub C:\dvad\outputs\submission.json --out
submissions\final.json
```

`--label-source {hybrid,appearance,rules}` controls who owns the label: `hybrid` lets the classifier decide and permits rules to *add* classes it cannot emit.

---

8. Evaluation

Local scorer

`src/score_arena.py` reproduces the official scoring mechanics: per-level marks, class matching, temporal IoU≥0.5, and false-alarm counting.

```
python src\score_arena.py --gt <ground_truth.csv> --sub <submission.json>
```

It reproduced a live total and a per-level score exactly, and every match/false-alarm count. **The counts are exact; the mark weights for the timed levels are estimated** and under-penalise false alarms relative to the real metric — trust the counts, not the decimal.

Supporting tools:

tool	question it answers
`src/diag_iou_gap.py`	every ground-truth interval vs our best same-class window, with IoU — is a miss near or nowhere?
`src/make_patch.py`	builds a partial submission and scores the merged result *before* spending an upload
`src/score_vlm_eval.py`	scores a Kaggle VLM dump against the banked sheet, per video
`src/merge_runtime.py`	keeps measured `runtime_metadata` when only events are re-exported

Cloud evaluation (Kaggle)

Local VLM inference is impractical, so bulk evaluation runs on a Kaggle T4. The videos are **not** uploaded — the model only ever sees decoded frames, so `src/build_eval_frames.py` packs sampled JPEGs instead, turning 1.27 GB of video into a 38 MB dataset.

Two sampling regimes, because the levels ask different questions:

- **Short clips** get motion-aware sampling — a transient event can fall between the points of a uniform grid.
- **Long clips** get a strictly **uniform time grid**, because their per-frame labels are rebuilt into intervals afterwards and a motion-biased grid would distort the very timeline it is meant to measure.

Frames are capped at 768 px on the long side to match how the fine-tuning set was built; feeding the adapter a different resolution is a silent distribution shift.

```
python src\build_eval_frames.py --videos <video_dir> --manifest <manifest.json> `
    --out C:\dvad\data\eval_frames_pack --push

python src\push_notebook.py --push --slug dvad-eval-frames-run `
    --notebook notebooks\eval_frames_kaggle.ipynb `
    --dataset dvad-eval-frames,dvad-vlm-ckpt400b4 --accelerator NvidiaTeslaT4

python src\push_notebook.py --pull --slug dvad-eval-frames-run --out
C:\dvad\models\kaggle_evalframes
```

Outputs `eval_frames.jsonl` (one row per frame with its timestamp — the product, for long clips) and `eval_results.jsonl` (per-video rollup).

---

9. Measured performance

Source: 4K 25 fps, GTX 1650 (4GB, no tensor cores).

configuration	throughput
Stage 1 alone, threaded decode	26.6 fps — 1.06x real-time on 4K
Stage 1 serial decode+detect	20.0 fps
Full pipeline, `--decision rules`, stride 2	**14.9 fps vs 12.5 needed**
`--decision hybrid` (3B VLM in-loop)	6.8 fps — not real-time on one card
Annotated 4K encode	drops 26.6 → 9.7 fps (hence `--save-width 1280`)

Event trigger rate: **0.19 – 1.9% of frames**, which is what makes the design viable on a card without tensor cores.

---

10. Engineering rules that held

- **Never promote a checkpoint or CSV without re-scoring it.** Every accepted improvement was independently verified first.
- **Never tune against the one test set you can see.** Several changes that looked strong in-sample were measurably worse held-out and were dropped. Using visible ground truth to *check* a decision is fine; using it to *fit* is the leak that fails an unseen set.
- **Never upload a full sheet to fix one video.** A submission only updates the videos it mentions, so `make_patch.py` scopes a change and scores the merged result first. It caught a regression that would have shipped alongside a genuine win.
- **Read composed output before shipping it.** Four self-contradiction classes in the explanations were caught by reading, not by tests.

- **Prefer a short working script to an abstracted framework.** Every file in `src/` runs standalone with `--help`.

---

11. Environment

item	value
Interpreter	`C:\dvad\.venv\Scripts\python.exe` (3.11, native Windows)
Code	this directory (OneDrive-synced, small files only)
Heavy artifacts	`C:\dvad\{models,data,outputs}` — never in OneDrive
GPU	GTX 1650, 4GB, sm_75, no tensor cores
Training	Kaggle T4 only — **never P100** (sm_60; 4-bit kernels are not built for it)
Local VLM runtime	Ollama for Windows (native CUDA, holds weights outside our RAM budget)

Two traps worth remembering:

- A `dataset-metadata.json` written by PowerShell carries a UTF-8 BOM, and the Kaggle CLI reports it as `Expecting value: line 1 column 1`. Write it with `Path.write_bytes()`.
- A Kaggle notebook title that does not slugify to the requested `--slug` silently changes the slug, and later status checks fail with a *permissions* error rather than "not found".